

GhostPass — documentation technique

Dossier technique — moyen de cryptologie GhostPass

Annexe technique destinée à la **déclaration de fourniture d'un moyen de cryptologie** auprès de l'ANSSI.

1. Objet

GhostPass est un gestionnaire de mots de passe à **connaissance nulle** : le serveur ne détient à aucun moment de quoi lire le contenu d'un coffre. Le chiffrement et le déchiffrement ont lieu sur l'appareil de l'utilisateur.

Il est distribué sous **Elastic License 2.0**, et peut être **auto-hébergé** : l'éditeur n'est pas un passage obligé.

Formes de mise à disposition : application iOS (App Store), application Android, application web, et serveur auto-hébergeable.

2. Fonctions cryptographiques mises en œuvre

Le cœur cryptographique est unique et écrit en Rust — `suite/crates/ghost-crypto` —, partagé par toutes les applications au moyen de liaisons générées (UniFFI pour Swift et Kotlin, WebAssembly pour le web). **Il n'existe pas de seconde implémentation**, et c'est une décision d'architecture : deux implémentations divergent, et la divergence ne se voit pas.

Fonction	Primitive	Paramètres	Fichier
Dérivation de clé depuis le mot de passe maître	Argon2id	64 Mio, 3 passes, parallélisme 4, sortie 32 octets	<code>kdf.rs</code>
Sel de dérivation	SHA-256 de l'adresse normalisée, tronqué	16 octets, déterministe	<code>kdf.rs</code>
Séparation des clés dérivées	HKDF-SHA-256	étiquettes <code>stackops:user-encryption-</code>	<code>kdf.rs</code>

Fonction	Primitive	Paramètres	Fichier
		key:v1 et stackops:master- password-auth:v1	
Chiffrement du coffre	XChaCha20-Poly1305	clé 256 bits, nonce 192 bits (24 octets)	symmetric.rs
Enveloppes de partage de secret	AES-256-GCM	clé 256 bits, nonce 96 bits (12 octets)	partage.rs
Scellement des clés d'organisation	crypto_box (X25519 + ChaCha20-Poly1305)	authentifié émetteur → destinataire	sharing.rs, org.rs
Accord de clés	X25519	—	keys.rs
Empreintes	SHA-256	—	kdf.rs

Le sel est **déterministe**, dérivé de l'adresse de courriel, afin que le client puisse dériver sa clé avant toute connexion — le serveur n'a donc pas à distribuer de sel, ni à savoir qui tente d'ouvrir.

Les paramètres du KDF reçus du serveur sont **revérifiés côté client** avant dérivation (KdfParams::ensure_strong, planchers 64 Mio et 3 passes). Sans ce contrôle, un serveur hostile pourrait imposer un coût trivial et rendre le condensat d'authentification attaquable hors ligne.

Bibliothèques tierces

argon2 0.5, chacha20poly1305 0.10, aes-gcm 0.10, crypto_box 0.9, x25519-dalek 2.0, hkdf 0.12, sha2 0.10, zeroize 1.9, rand_core 0.6 (entropie fournie par le système d'exploitation via getrandom). Ce sont des implémentations publiques et auditables ; **aucun algorithme n'a été écrit pour ce produit.**

3. Gestion des clés

Le mot de passe maître ne quitte jamais l'appareil. De lui sont dérivées, par HKDF sur des étiquettes distinctes, deux clés qui ne se recouvrent pas :

- une **clé de chiffrement**, qui protège la clé symétrique de l'utilisateur et **reste sur le client** ;
- un **condensat d'authentification**, seul à être transmis, et que le serveur recondense avant stockage.

Chaque élément du coffre porte sa propre clé, elle-même chiffrée sous la clé de l'utilisateur. Les coffres d'organisation emploient une clé d'organisation scellée individuellement pour chaque membre par une box authentifiée.

Conservation sur l'appareil : sur iOS, trousseau système avec kSecAttrAccessibleWhenPasscodeSetThisDeviceOnly et, pour le déverrouillage biométrique, .biometryCurrentSet — l'entrée est invalidée dès qu'une biométrie est

enrôlée. Sur Android, AndroidKeyStore avec setUserAuthenticationRequired, biométrie forte seule, setInvalidatedByBiometricEnrollment et setUnlockedDeviceRequired. Les secrets en mémoire sont effacés (zeroize).

4. Ce que le serveur détient — et ce qu’il ne détient pas

Table users : adresse de courriel, paramètres KDF, condensat d’authentification recondensé, sel, **clé de l’utilisateur chiffrée**, **clé privée chiffrée**, clé publique, et le cas échéant le secret du second facteur et les éléments de récupération.

Table vault_items : identifiants, **clé chiffrée**, **données chiffrées**, horodatages.

Le serveur détient donc des adresses de courriel et des blocs chiffrés. Il **ne détient pas** : mot de passe maître, clé de chiffrement, clé privée en clair, ni aucun contenu de coffre en clair. Il ne peut pas les reconstituer.

Cette distinction est celle qui figure dans la politique de confidentialité publiée : qu’une donnée soit illisible ne la rend pas inexistante, et la déclaration ne prétend pas « aucune donnée collectée ».

5. Transport

HTTPS/TLS entre les applications et le serveur, assuré par l’hébergement. Le chiffrement de bout en bout est **indépendant** du transport : il protège le contenu y compris d’un serveur compromis, ce que TLS ne fait pas.

6. Interopérabilité

Formats et vecteurs figés dans suite/assets/vecteurs/contrat.json, éprouvés par des témoins croisés entre les implémentations Rust, Swift, Kotlin et WebCrypto. L’enveloppe de partage emploie **AES-256-GCM** plutôt que XChaCha20 pour une raison précise : le partage s’ouvre dans un navigateur, et WebCrypto ne fournit pas XChaCha20.